
Tutorial: Building a Simple Content-Based Recommender System

Step 1: Imports Import Libraries

Let's start by importing the required libraries.

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import linear_kernel
```

Step 2: Data Loading

Load the movie and ratings data using Pandas.

```
movies = pd.read_csv('movies.csv')
```

Or if you're using Google Colab, you need to mount your Google Drive to access the dataset.

```
from google.colab import drive
drive.mount('/content/gdrive')
```

```
movies = pd.read_csv('gdrive/MyDrive/RS/movielens-20m-dataset/movie.csv')
ratings = pd.read_csv('gdrive/MyDrive/RS/movielens-20m-dataset/rating.csv')
```

Step 3: Preprocess Data

In this step, we'll preprocess the 'genres' column by replacing '|' with a space.

movied	title	genres		
1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy		
2	Jumanji (1995)	Adventure Children Fantasy		
3	Grumpier Old Men	Comedy Romance		
4	Waiting to Exhale	Comedy Drama Romance		
5	Father of the Bride	Comedy		
6	Heat (1995)	Action Crime Thriller		
7	Sabrina (1995)	Comedy Romance		
8	Tom and Huck	Adventure Children		
9	Sudden Death	Action		
10	GoldenEye (1995)	Action Adventure Thriller		

```
movies['genres'] = movies['genres'].apply(lambda x: x.replace('|', ' '))
```

Step 3 TF-IDF (Term Frequency-Inverse Document Frequency):

Transform movie genres into a TF-IDF matrix representation. TF-IDF stands for Term Frequency-Inverse Document Frequency and helps measure word importance.

TF-IDF represents how important a word is to a document in a collection (corpus). It consists of two parts:

TF (Term Frequency) and IDF (Inverse Document Frequency).

- Term Frequency (TF): Measures how often a word appears in a document.
- Inverse Document Frequency (IDF): Measures how unique or rare a word is across all documents in the corpus.

Diagram:

Document 1	Document 2	...	Document N
TF-IDF	TF-IDF		
Vector	Vector		

Table:

Word	TF-IDF Value (Document 1)	...	TF-IDF Value (Document N)
love	0.05	...	0.02
action	0.12	...	0.09
drama	0.08	...	0.03
...	...	...	...

The TF-IDF value of a word in a document increases with its frequency in the document and decreases with its frequency in the entire corpus.

TF-IDF Formula: $TF-IDF = TF * IDF$

Calculating TF-IDF with Libraries:

In our example, we'll use the `TfidfVectorizer` from the `sklearn.feature\_extraction.text` module to calculate TF-IDF values for movie genres.

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Initialize the TF-IDF vectorizer
tfidf_vectorizer = TfidfVectorizer(stop_words='english')

# Fit and transform the genres column
tfidf_matrix = tfidf_vectorizer.fit_transform(movies['genres'])
```

Now, let's break down the process with a step-by-step explanation and a simple example.

Example Calculation:

Consider two documents (movies) and a simplified vocabulary:

Document A: "action adventure"

Document B: "action drama"

Vocabulary: ["action", "adventure", "drama"]

1. Term Frequency (TF):

Calculate the term frequency for each term in the documents.

- $TF(\text{"action"}, \text{Document A}) = 1 / 2 = 0.5$
- $TF(\text{"adventure"}, \text{Document A}) = 1 / 2 = 0.5$
- $TF(\text{"action"}, \text{Document B}) = 1 / 2 = 0.5$
- $TF(\text{"drama"}, \text{Document B}) = 1 / 2 = 0.5$

2. Inverse Document Frequency (IDF):

Calculate the inverse document frequency for each term.

- Number of documents (corpus size) = 2
- $IDF(\text{"action"}) = \log(2 / 2) = 0$
- $IDF(\text{"adventure"}) = \log(2 / 1) = 0.693$
- $IDF(\text{"drama"}) = \log(2 / 1) = 0.693$

3. TF-IDF Calculation:

Calculate the TF-IDF value for each term in the documents.

- $TF-IDF(\text{"action"}, \text{Document A}) = 0.5 * 0 = 0$
- $TF-IDF(\text{"adventure"}, \text{Document A}) = 0.5 * 0.693 = 0.3465$
- $TF-IDF(\text{"action"}, \text{Document B}) = 0.5 * 0 = 0$
- $TF-IDF(\text{"drama"}, \text{Document B}) = 0.5 * 0.693 = 0.3465$

This is a simplified example, but it demonstrates how TF-IDF values are calculated for terms within documents.

In our content-based recommender system, we apply the TF-IDF vectorization to movie genres. The TF-IDF matrix represents how important each genre term is within each movie's genre profile. This matrix is then used to calculate cosine similarity, which helps us identify similar movies based on their genre profiles. The higher the TF-IDF values for shared terms between two movies, the more similar their genre profiles are.

Step 4: Cosine Similarity

Calculate the cosine similarity between movies based on their TF-IDF vectors.

Cosine similarity measures the cosine of the angle between two non-zero vectors, indicating how similar they are in direction. In the context of text data, it helps us find the similarity between movie genre vectors.

Movie 1	Movie 2	...	Movie N
-----	-----	-----	-----
Cosine Sim	Cosine Sim		
(0.92)	(0.78)		
-----	-----	-----	-----

In the context of text data like movie genres, it's used to determine how similar the genre profiles of two movies are. The cosine similarity ranges from -1 to 1, where:

- 1 implies the vectors are identical.
- 0 implies the vectors are orthogonal (not similar).
- -1 implies the vectors are diametrically opposed.

Mathematically, the cosine similarity between two vectors A and B is calculated using the dot product and magnitudes of the vectors:

$$\text{Cosine Similarity (A, B)} = (A \cdot B) / (||A|| * ||B||)$$

Where:

- $A \cdot B$ is the dot product of vectors A and B.
- $||A||$ represents the magnitude (Euclidean norm) of vector A.
- $||B||$ represents the magnitude of vector B.

Let's consider two movies with genre vectors A and B, represented as TF-IDF values. For simplicity, we'll assume they have only two genre terms: 'action' and 'adventure'.

Movie A: [0.2, 0.8] (TF-IDF values)

Movie B: [0.6, 0.4] (TF-IDF values)

1. Calculate the dot product of A and B:

$$A \cdot B = (0.2 * 0.6) + (0.8 * 0.4) = 0.12 + 0.32 = 0.44$$

2. Calculate the magnitudes of A and B:

$$||A|| = \sqrt{(0.2^2 + 0.8^2)} \approx 0.8367$$

$$||B|| = \sqrt{(0.6^2 + 0.4^2)} \approx 0.7211$$

3. Plug these values into the formula:

$$\text{Cosine Similarity (A, B)} = 0.44 / (0.8367 * 0.7211) \approx 0.7725$$

In our example, the cosine similarity between movies A and B is approximately 0.7725, indicating that they have a moderate degree of similarity in terms of their genre profiles.

Calculating Cosine Similarity with Libraries:

In our content-based recommender system example, the `linear\_kernel` function from the `sklearn.metrics.pairwise` module is used to calculate the cosine similarity between all pairs of movies represented as TF-IDF vectors.

```
from sklearn.metrics.pairwise import linear_kernel

# Calculate cosine similarity between movies
cosine_sim = linear_kernel(tfidf_matrix, tfidf_matrix)
```

This `cosine\_sim` matrix holds the cosine similarity scores between each movie pair. You can then use these similarity scores to recommend movies that are most similar to a given movie.

Cosine similarity is a powerful concept in recommendation systems, allowing you to measure similarity between items (movies in this case) based on their features (genre vectors). It's an essential component in content-based recommenders and has applications in various fields beyond recommendation systems.

Step 5: Define Recommender Function

Create a function that takes a movie title as input and returns a list of recommended similar movies. The recommender function uses the cosine similarity matrix to find movies that are most similar to the input movie based on their genre vectors.

Step 6: Test Recommender System

Test your recommender system by providing a movie title and getting recommendations. The recommender system will return a list of movies similar to the input movie based on their genres.

By following these steps, you'll be able to implement a content-based recommender system that suggests movies to users based on the similarity of movie genres. The process involves preprocessing data, transforming it into a TF-IDF matrix, calculating cosine similarity, and defining a function to generate recommendations.